Data Bot AI v1.0 — Engineering Handbook

README.md

Data Bot AI v1.0 — Engineering Handbook

Official documentation package for Data Bot AI version **1.0.0**.

This handbook is simultaneously:

| Role | Audience |

| Software Engineering Documentation | Engineers |

| AI Architecture Documentation | AI/ML engineers |

| Data Science & Statistics Notes | Analysts |

| Power BI Learning Guide | BI practitioners |

| System Design Documentation | Architects |

| Portfolio Case Study | Hiring managers |

| Interview Preparation Guide | Candidates |

| Enterprise / Onboarding Manual | New team members |

| Technical Evidence | Auditors / reviewers |

Version & Evidence

| Item | Value | Source |

| Product version | `1.0.0` | `backend/core/config.py` |

| App name | AI Analytics SaaS MVP (Data Bot AI) | settings |

| Test suite (release gate) | **634 passed** | Release verification 2026-07-10 |

| Test files (`test_*.py`) | **125** | `tests/` |

| Backend services (`*_service.py`) | **89** | `backend/services/` |

| API route modules | **28** | `backend/api/routes/` |

| Streamlit page modules | **34+** | `frontend/app_pages/` |

| Domain models | **49** | `backend/models/` |

| Git tag | `v1.0.0` | GitHub release |

How to read this handbook

1. Start with [01 Executive Summary](01_executive_summary/README.md)

2. Study [02 Architecture](02_architecture/README.md)

3. Drill into folders, workflows, API, frontend as needed

4. Use Learning / Interview / Portfolio sections for career prep

Documentation standards

See [20 Standards](20_standards/README.md).

Rule: Never invent features. Items not confirmed in the repository are labeled **Not verified**.

Export formats

See [19 Formats](19_formats/README.md) and `scripts/export_handbook.py`.

Repository map (top level)

ai-analytics-saas-mvp/

■■■ backend/ # FastAPI application

■■■ frontend/ # Streamlit UI

■■■ tests/ # pytest suite

■■■ docs/ # ADR + release guides

■■■ documentation/ # THIS handbook (v1.0 master docs)

■■■ release/ # v1.0 / v1.0-rc checklists

■■■ docker/ # Compose + Dockerfiles

■■■ deploy/ # Deploy scripts

■■■ alembic/ # DB migrations

■■■ data/ # Local data (samples committed; runtime data gitignored)

■■■ requirements*.txt # Pinned dependencies

01_executive_summary / README.md

01 — Executive Summary

What is Data Bot AI?

Data Bot AI (repository product name: **AI Analytics SaaS MVP**) is a **local-first analytics and AI analyst platform**. Users upload CSV/Excel datasets, explore KPIs and charts, run natural-language analysis through an AI Analyst runtime, manage knowledge/RAG, execute workflows and background jobs, and operate the platform with auth, RBAC, storage, monitoring, and commercial controls.

Verified version: `1.0.0` (`backend/core/config.py`, GitHub tag `v1.0.0`).

Business problem solved

Organizations need a single place to:

1. Ingest tabular business data without a heavy BI stack

2. Produce dashboards, KPIs, and executive narratives quickly

3. Ask questions in natural language with governed AI workflows

4. Keep multi-tenant identity, storage, jobs, and observability in one product

Target users

| Persona | Needs (verified by UI/API surface) |

| Business analyst | Upload, clean, dashboard, charts, reports |

| Data / AI practitioner | AI Analyst, workflows, knowledge, evaluation |

| Platform admin | Orgs, RBAC, health, metrics, admin dashboard |

| Commercial operator | Plans, usage, API keys, invoices (no live payment gateway) |

Major capabilities (verified)

- Dataset upload (CSV/XLSX), cleaning, profiling, analytics dashboards

- Charts, pivot, visual builder, SQL Lab, DAX Studio

- Reports (PDF/PPT), storyboard, geospatial/location insights

- AI Analyst (`/api/v1`), workflows, evaluation, knowledge ingestion

- RAG-related services and `/rag` route module (**mount in `main.py`: Not verified** — `rag_routes.py` exists; not listed in `create_app()` includes as of v1.0.0)

- Auth (JWT), organizations, workspaces, RBAC

- Jobs/queue/workers, object storage lifecycle

- Monitoring, metrics, release validation

- Billing plans/usage/API keys/admin (in-memory commercial stores for MVP)

Architecture overview

flowchart TB

UI[Streamlit Frontend]

API[FastAPI backend.main]

SVC[Service Layer]

REPO[Repositories]

DB[(SQLite/Postgres)]

STORE[Object Storage local/S3 stub]

Q[Queue memory/Redis]

AI[AI Runtime: LLM Agents RAG Workflow]

UI -->|HTTP JSON| API

API --> SVC

SVC --> REPO --> DB

SVC --> STORE

SVC --> Q

SVC --> AI

Technology stack (pinned)

| Layer | Technology | Pin evidence |

| API | FastAPI 0.115.6, Uvicorn 0.32.1 | `requirements.txt` |

| UI | Streamlit 1.40.2 | `requirements.txt` |

| Data | pandas 2.2.3, Plotly 6.8.0 | `requirements.txt` |

| ORM | SQLAlchemy 2.0.50, Alembic 1.14.0 | `requirements.txt` |

| Queue | redis 5.2.1 (optional) | `requirements.txt` |

| Export | reportlab, python-pptx, openpyxl, Pillow, kaleido, matplotlib | `requirements.txt` |

| Auth crypto | Custom JWT/HMAC + PBKDF2 (stdlib) | `backend/security/` |

System lifecycle

1. **Develop** on `main`

2. **Test** with pytest (634 passed at v1.0.0 gate)

3. **Deploy** API (`uvicorn backend.main:app`) + Streamlit + optional worker

4. **Operate** via `/api/v1/live`, `/ready`, monitoring, release validation

5. **Maintain** hotfixes on `release/v1.0`

Deployment model

- Single-node / Docker Compose (`docker/`)

- Local filesystem storage by default

- Optional PostgreSQL + Redis

- **Not verified / out of scope for 1.0:** Kubernetes manifests, live billing gateway, enterprise SSO

Advantages

- End-to-end product surface (data → AI → ops → commercial)

- Local-first MVP with production hardening (Sprint 8.7)

- Large automated test suite

- Documented release gates and known limitations

Limitations

See root `KNOWN_ISSUES.md` (KI-001…KI-008): no payment gateway, some in-memory stores, S3 stub, no K8s, no SSO, permissive CORS defaults, JWT secret must be set in production.

Future roadmap

See root `ROADMAP.md`: billing gateway, SQL commercial stores, S3 completion, K8s, SSO, forecast plugins, UI polish.

02_architecture / README.md

02 — Architecture Handbook

Layer diagram

flowchart TB

subgraph Presentation

ST[Streamlit pages + clients]

end

subgraph API

R[Route modules]

MW[Middleware: CORS GZip Security RateLimit CSRF Monitoring Auth]

end

subgraph Domain

S[Services ~89 modules]

M[Pydantic / domain models]

end

subgraph Infrastructure

REP[Repositories memory/SQLAlchemy]

STG[Storage providers]

QUE[Queue backends]

MON[Monitoring + Logging]

REL[Reliability: circuit retry timeout shutdown]

PERF[Performance: cache pagination streaming]

end

ST --> R --> S

R --> MW

S --> M

S --> REP

S --> STG

S --> QUE

S --> MON

S --> REL

S --> PERF

Middleware order (`backend/main.py`)

Last added = outermost. Approximate request path:

1. CORSMiddleware (env-driven origins)

2. GZipMiddleware (min 500 bytes)

3. SecurityHeadersMiddleware

4. RateLimitMiddleware

5. CSRFMiddleware (optional via `CSRF_ENABLED`)

6. MonitoringMiddleware

7. AuthContextMiddleware (non-blocking bearer attach)

Lifespan: graceful shutdown via `backend.reliability.shutdown`.

Frontend architecture

- Entry: `frontend/streamlit_app.py`

- Navigation: `_NAV_GROUPS` sidebar expanders (Home, AI Workspace, Data, Analytics, AI Legacy, Reports, Advanced, Account, Administration, Commercial, Operations)

- Pages: `frontend/app_pages/*_page.py`

- API clients: `frontend/api/*_client.py` + legacy `frontend/api_client/`

- Session: `frontend/utils/session_state.py`, `auth_state.py`

Backend packages (verified folders)

| Package | Purpose |

| `api/` | Routes, middleware, auth deps, error handlers |

| `services/` | Business logic |

| `models/` | Domain schemas |

| `database/` | Engine, session, SQLAlchemy models |

| `repositories/` | Persistence abstraction |

| `storage/` | Object storage abstraction |

| `queue/` + `jobs/` + `workers/` | Async execution |

| `monitoring/` + `logging/` | Observability |

| `security/` | JWT, passwords, hardening |

| `performance/` + `reliability/` | Production readiness |

| `config/` | Typed settings loader |

| `processing/` | Cleaning, profiling, schema |

| `registry/` | Domain/KPI/visualization registries |

| `ai/`, `rag/`, `storyboard/` | Supporting AI/BI packages |

AI Analyst Runtime (verified services)

Key modules (exist under `backend/services/`):

- `ai_analyst_service.py`, `ai_analyst_runtime_service.py`

- `planning_service.py`, `tool_selection_service.py`, `tool_registry_service.py`

- `agent_service.py`, `memory_service.py`

- `rag_service.py`, `embedding_service.py`, `vector_store_service.py`, `context_retrieval_service.py`

- `workflow_engine_service.py`

- `evaluation_service.py`, `ai_validation_service.py`, `output_validation_service.py`

- `llm_service.py` + `providers/` (openai, anthropic, local)

Auth / RBAC / Orgs

- Auth: `auth_service.py`, `security/jwt_service.py`, `password_service.py`, routes `/api/v1/auth`

- Orgs/workspaces: `organization_service.py`, `workspace_service.py`

- RBAC: `rbac_service.py` (`evaluate_access`, `has_permission`)

Jobs / Storage / Monitoring / Commercial

- Jobs: `job_service.py`, queue factory (memory/redis), workers CLI

- Storage: `storage_service.py`, local provider, S3 stub

- Monitoring: health, metrics, tracing, collectors

- Commercial: `billing_service`, `subscription_service`, `usage_service`, `api_key_service`, `admin_service`

Sequence: authenticated API call

sequenceDiagram

participant U as Streamlit

participant M as Middleware

participant R as Route

participant S as Service

U->>M: Bearer token + request

M->>M: Rate limit / CSRF / headers / metrics

M->>R: Auth context attached if valid

R->>S: Domain operation

S-->>R: Result / AuthError / ServiceError

R-->>U: JSON response

Dependency diagram (high level)

flowchart LR

streamlit --> fastapi

fastapi --> services

services --> sqlalchemy

services --> pandas

services --> storage

services --> queue

services --> llm_providers

Folder diagram

See [03 Folder Documentation](../03_folders/README.md).

Class diagrams

Practical class diagrams are limited because many stores are dict-backed services rather than rich OOP hierarchies. Repository interfaces live in `backend/repositories/interfaces.py` with memory and SQLAlchemy implementations — see Database Handbook.

02_architecture / COMPONENT_CATALOG.md

Component Catalog

API route modules (28)

Module	Prefix (from source)	Mounted in main.py
upload_routes	(upload)	Yes
dataset_routes	`/datasets`	Yes
analytics_routes	`/analytics`	Yes
insight_routes	`/insights`	Yes
intelligence_routes	`/intelligence`	Yes
visual_builder_routes	`/visual-builder`	Yes
report_routes	`/report`	Yes
theme_routes	`/themes`	Yes
branding_routes	`/branding`	Yes
sql_lab_routes	`/sql-lab`	Yes
dax_routes	`/dax`	Yes
system_routes	`/api/v1`	Yes
analyst_routes	`/api/v1`	Yes
workflow_routes	`/api/v1`	Yes
evaluation_routes	`/api/v1`	Yes
knowledge_routes	`/api/v1`	Yes
auth_routes	`/api/v1/auth`	Yes
organization_routes	`/api/v1/organizations`	Yes
workspace_routes	`/api/v1/workspaces`	Yes
rbac_routes	`/api/v1`	Yes
job_routes	`/api/v1/jobs`	Yes
storage_routes	`/api/v1/storage`	Yes
monitoring_routes	`/api/v1`	Yes
billing_routes	`/api/v1/billing`	Yes
apikey_routes	`/api/v1/api-keys`	Yes
admin_routes	`/api/v1/admin`	Yes
release_routes	`/api/v1/release`	Yes
rag_routes	`/rag`	**Not verified** (file exists; not in `create_app()` includes)

Service domains (selected)

See `backend/services/` for the full list of 89 `*_service.py` files spanning analytics, AI, forecast, commercial, and platform domains.

03_folders / README.md

03 — Folder Documentation

Top-level

| Folder | Purpose | Inputs | Outputs | Extension points |

| `backend/` | FastAPI app + domain logic | HTTP, env, files | JSON, files, metrics | New routes/services |

| `frontend/` | Streamlit UI | User actions, API | Rendered pages | New pages/clients |

| `tests/` | Automated verification | Fixtures, TestClient | Pass/fail | New test modules |

| `docs/` | ADRs + release guides | Authors | Markdown | New ADRs |

| `documentation/` | This Engineering Handbook | Repo analysis | Handbook | New chapters |

| `release/` | Checklists for v1.0 / RC | Release process | Gate evidence | Patch checklists |

| `docker/` | Compose + Dockerfiles | Env template | Containers | New services |

| `deploy/` | Deploy helper scripts | Ops | Process starts | New scripts |

| `alembic/` | SQL migrations | Models | Schema versions | New revisions |

| `data/` | Runtime + samples | Uploads | Datasets/storage | New samples |

| `scripts/` | Tooling (handbook gen/export) | — | Artifacts | New tools |

Important backend files

| Path | Responsibility |

| `backend/main.py` | App factory, middleware, router includes, lifespan |

| `backend/core/config.py` | Paths, version, upload limits, ensure dirs |

| `backend/api/error_handlers.py` | HTTP error mapping |

| `backend/api/auth_dependencies.py` | Current user / permission deps |

| `backend/database/session.py` | Engine + pooling + session scope |

| `backend/repositories/registry.py` | Backend switching memory/SQL |

Important frontend files

| Path | Responsibility |

| `frontend/streamlit_app.py` | Entry, nav groups, page routing |

| `frontend/utils/session_state.py` | Session keys |

| `frontend/utils/auth_state.py` | Auth session helpers |

| `frontend/api/*.py` | Typed HTTP clients |

Best practices

- Keep business logic in services, not routes

- Prefer repository interfaces for persistence

- Paginate list endpoints (`performance.pagination`)

- Do not commit secrets or `data/*.db`

04_workflows / README.md

04 — Workflow Documentation

1) Dataset Upload

sequenceDiagram

participant U as User/UI

participant API as POST /upload

participant US as upload_service

participant DS as dataset_service

participant ST as storage (optional)

U->>API: CSV/XLSX multipart

API->>US: validate + store

US->>DS: create_dataset / metadata

DS-->>API: dataset_id + status ready

API-->>U: JSON

- **Inputs:** file bytes, filename, extension in `{.csv,.xlsx,.xlsm}`

- **Outputs:** `dataset_id`, status, row/column counts

- **Errors:** invalid type/size (MAX_UPLOAD_SIZE_MB)

- **Services:** `upload_service`, `dataset_service`, cleaning/profiling downstream

2) AI Analysis (AI Analyst)

- **Entry:** `/api/v1` analyst routes + Streamlit AI Analyst page

- **Services:** `ai_analyst_service`, runtime, planning, tools, memory, LLM providers

- **Outputs:** session artifacts, insights, optional evaluation

- **State:** session history page resumes prior sessions

3) Workflow Execution

- **Engine:** `workflow_engine_service.execute_workflow`

- **API:** `/api/v1` workflow routes

- **UI:** Workflow Monitor

- **Jobs:** can be submitted async via `job_service`

4) Knowledge Retrieval

- **Ingestion:** `knowledge_ingestion_service` + knowledge routes

- **Retrieval:** RAG/context services (`rag_service`, `context_retrieval_service`, embeddings, vector store)

- **Note:** `rag_routes.py` exists; **mount in main.py Not verified**

5) Evaluation

- **Service:** `evaluation_service` (session/workflow/agent/tool/RAG/LLM metrics)

- **API:** evaluation routes under `/api/v1`

- **UI:** Evaluation Dashboard

6) Authentication

stateDiagram-v2

[*] --> Anonymous

Anonymous --> Registered: POST /auth/register

Registered --> Authenticated: POST /auth/login

Authenticated --> Locked: brute-force threshold

Locked --> Authenticated: lockout expiry + success

Authenticated --> Anonymous: logout / token expiry

- Brute-force: `backend/security/brute_force.py` integrated in `authenticate_user`

- Tokens: access + refresh (custom JWT)

7) Storage

- Upload/list/download/archive/restore/rollback/verify

- Pagination on list; streaming download `?stream=true`

- Versioning + retention helpers in `backend/storage/`

8) Background Jobs

- Submit → queue → worker/inline execute → status/progress → retry/cancel

- Types include workflow_execution, analysis, evaluation, knowledge_ingestion, generic (per job routes docs)

9) Monitoring

- Liveness `/api/v1/live`, readiness `/api/v1/ready`, health, metrics, system status

- Release: `/api/v1/release/validation`, benchmarks, security audit, performance, recovery

Retry / circuit / timeout

Implemented as abstractions in `backend/reliability/` (retry, circuit breaker, timeouts, fallback, shutdown). Job retries also exist in job service. Exact production wiring of circuit breakers into every external call: **partially verified** (benchmarks/release use circuits; not every service call).

05_data_science / README.md

05 — Data Science Handbook

This chapter explains Data Bot AI from a data-science lens using **verified** product surfaces.

Data Cleaning

- **Why:** Raw CSV/Excel contains nulls, duplicates, type noise

- **Where:** `data_cleaning_service`, processing `data_cleaner`, UI Data Cleaning

- **Business value:** Trustworthy KPIs and charts

- **Limitation:** Pandas `object` dtype warnings documented in KI-008

EDA / Profiling / Schema

- Overview endpoints on datasets: row/column counts, column groups (numeric/categorical), missingness, preview

- Services: `overview_service`, `data_profiler`, `schema_service`, `column_detector`

KPI Detection & Business Metrics

- `kpi_service`, dashboard KPI cards with sparklines, reasons, recommended actions (verified in dataset phase tests historically)

- Domain intelligence: `domain_intelligence_service`, registries under `backend/registry/`

Executive Insights & Storytelling

- Insight services, executive reasoning, storyboard engine, PDF/PPT export services

- UI: AI Insights, Storyboard, Reports

Forecast Architecture

Verified forecast-related services exist:

- adapter, capability, dataset, explainability, governance, pipeline, plugin, scenario

- Prediction engine + prediction validation

Treat forecast depth as **modular MVP architecture**; production model accuracy claims: **Not verified**.

Validation & Evaluation

- AI validation, output validation, evaluation metrics across workflow/agents/tools/RAG/LLM

- Business value: reduce hallucinations and measure quality

Decision Intelligence & Memory

- `decision_intelligence_service`, `memory_service`

- Supports analyst continuity across sessions

Knowledge Retrieval / Business AI

- Knowledge ingestion + RAG stack services

- LLM providers: OpenAI, Anthropic, local stubs/adapters under `providers/`

Practical scenario

Sales CSV (`data/samples/sample_sales_data.csv`) → upload → overview → dashboard KPIs (sales/profit) → AI Analyst question → optional evaluation score.

06_statistics / README.md

06 — Statistics Handbook

Concepts commonly used by analytics platforms like Data Bot AI. Where the repo uses them is noted; otherwise labeled **general teaching**.

Descriptive statistics

| Concept | Definition | Formula (simple) | Business meaning | Likely usage in Data Bot AI |

| Mean | Average | Σx / n | Typical value | KPI totals/averages on dashboards |

| Median | 50th percentile | middle value | Robust center | Outlier-resistant summaries (**Not verified** exact median API) |

| Mode | Most frequent | argmax freq | Common category | Categorical profiling |

| Variance / Std Dev | Spread | $\Sigma(x-\mu)^2/(n-1)$; $\sqrt{var}$ | Volatility | Distribution charts |

| Percentiles / Quartiles | Ranked cut points | order stats | Segmentation | Profiling / charts |

| Correlation | Linear association | Pearson r | Co-movement | Charts / correlation views |

| Outliers | Extreme points | IQR/z-score heuristics | Data quality | Cleaning flows |

Inferential / forecasting (teaching + project touchpoints)

| Topic | Teaching point | Project touchpoint |

| Regression | Predict continuous Y from X | Forecast/prediction services exist |

| Confidence intervals | Uncertainty of estimate | Explainability services (**depth Not verified**) |

| Hypothesis testing | Compare groups | **Not verified** as first-class product feature |

| Time series / trend | Order by time | Sample sales has `date`; forecast pipeline |

| Forecast error | MAE/RMSE/MAPE | Evaluation/scoring services |

Interview questions (samples)

1. Mean vs median when outliers exist?

2. What does correlation not imply?

3. How would you detect outliers before KPI calculation?

4. Which error metric for forecasting revenue?

Power BI / Python mapping

- Power BI: DAX `AVERAGE`, `MEDIAN`, `STDEV.P`, visuals

- Python: `pandas` describe/corr; used heavily in backend processing

07_power_bi / README.md

07 — Power BI Mapping

| Data Bot AI capability | Power BI analogue | Notes |

| Upload + profiling | Power Query / Data view | Local CSV/XLSX first |

| Data cleaning | Power Query transforms | Nulls/duplicates/outliers |

| Dashboard KPIs | Cards + measures | `kpi_service` / analytics dashboard |

| Charts | Report visuals | Plotly in Streamlit |

| Dashboard Studio | Report canvas | Visual builder routes |

| DAX Studio page | DAX measures | `/dax` routes + `dax_service` |

| SQL Lab | DirectQuery / SQL | `/sql-lab` |

| AI Insights / Analyst | Smart Narrative / Copilot-like | Different stack (LLM services) |

| Storyboard / PPT-PDF | Paginated / export | reportlab / python-pptx |

| Themes / branding | Theme JSON | `/themes`, `/branding` |

| Evaluation | Performance Analyzer (loose) | Quality metrics, not UI perf |

| RBAC / orgs | Workspace roles | Custom RBAC API |

| Monitoring | Admin monitoring | Health/metrics endpoints |

| Governance docs | Tenant governance | Security + DR guides |

Example: KPI

Power BI: `Total Sales = SUM(Sales[Amount])`

Data Bot AI: dashboard KPI cards derived from dataset analytics services after upload.

08_ai / README.md

08 — AI Handbook

Beginner view

An **LLM** answers in natural language. Data Bot AI wraps LLMs with **planning**, **tools**, **memory**, **RAG**, and **evaluation** so answers stay closer to business data.

Advanced view (verified modules)

flowchart TB

Q[User query] --> P[planning_service]

P --> T[tool_selection + tool_registry]

T --> A[agent_service / analyst runtime]

A --> M[memory_service]

A --> R[rag_service + embeddings + vector_store]

A --> L[llm_service + providers]

A --> V[ai_validation / output_validation]

A --> E[evaluation_service]

Topics

| Topic | Repo evidence | Notes |

| Prompt engineering | `prompt_service`, prompt templates | |

| Planning / reasoning | `planning_service`, executive reasoning | |

| Agents / tools | `agent_service`, tool registry/selection | |

| Memory | `memory_service` | |

| Embeddings / vector search | `embedding_service`, `vector_store_service` | |

| RAG / knowledge | `rag_service`, knowledge ingestion | `/rag` mount Not verified |

| Evaluation / explainability | evaluation + forecast explainability | |

| Hallucination prevention | validation services | Not a guarantee |

| Providers | openai, anthropic, local | Keys via env (**ops**) |

| Governance | security guides, validation, RBAC | No enterprise SSO in 1.0 |

Safety

- Validate outputs before executive presentation

- Prefer retrieval-grounded answers when knowledge is ingested

- Keep secrets out of prompts/logs

09_api / README.md

09 — API Handbook

Base application: FastAPI (`backend/main.py`).

Legacy product routes often lack `/api/v1` prefix; platform routes use `/api/v1`.

OpenAPI UI: `/docs` when server is running (**verified FastAPI default**).

Authentication

- Register/login under `/api/v1/auth`

- Send `Authorization: Bearer <access_token>` for protected routes

- CSRF optional via `CSRF_ENABLED` + `X-CSRF-Token`

Endpoint inventory (auto-extracted decorators)

`admin_routes.py` — prefix `/api/v1/admin`

- `GET` `/api/v1/admin/dashboard`

- `GET` `/api/v1/admin/statistics`

- `GET` `/api/v1/admin/users`

- `GET` `/api/v1/admin/organizations`

- `GET` `/api/v1/admin/workspaces`

- `GET` `/api/v1/admin/subscriptions`

- `GET` `/api/v1/admin/api-keys`

- `GET` `/api/v1/admin/usage`
- `GET` `/api/v1/admin/audit`
- `GET` `/api/v1/admin/invoices`
- `GET` `/api/v1/admin/features`
- `PUT` `/api/v1/admin/features/{feature_key}`

`analyst_routes.py` — prefix `/api/v1`

- `POST` `/api/v1/analyst/analyze`
- `POST` `/api/v1/session/create`
- `GET` `/api/v1/session/{session_id}`
- `GET` `/api/v1/session/{session_id}/summary`
- `GET` `/api/v1/session/{session_id}/evaluation`
- `POST` `/api/v1/session/{session_id}/execute`

`analytics_routes.py` — prefix `/analytics`

- `GET` `/analytics/{dataset_id}/summary`
- `GET` `/analytics/{dataset_id}/dashboard`
- `POST` `/analytics/{dataset_id}/dashboard/filter`

`apikey_routes.py` — prefix `/api/v1/api-keys`

- `POST` `/api/v1/api-keys`
- `GET` `/api/v1/api-keys`
- `GET` `/api/v1/api-keys/{key_id}`
- `DELETE` `/api/v1/api-keys/{key_id}`
- `POST` `/api/v1/api-keys/{key_id}/rotate`

`auth_routes.py` — prefix `/api/v1/auth`

- `POST` `/api/v1/auth/register`
- `POST` `/api/v1/auth/login`
- `POST` `/api/v1/auth/refresh`

- `POST` `/api/v1/auth/logout`
- `POST` `/api/v1/auth/change-password`
- `POST` `/api/v1/auth/request-password-reset`
- `POST` `/api/v1/auth/reset-password`
- `POST` `/api/v1/auth/verify-email`
- `GET` `/api/v1/auth/me`
- `PUT` `/api/v1/auth/me/profile`
- `GET` `/api/v1/auth/sessions`

`billing_routes.py` — prefix `/api/v1/billing`

- `GET` `/api/v1/billing/plans`
- `GET` `/api/v1/billing/plans/{plan_id}`
- `GET` `/api/v1/billing/subscriptions/{organization_id}`
- `POST` `/api/v1/billing/subscriptions/{organization_id}`
- `POST` `/api/v1/billing/subscriptions/{organization_id}/upgrade`
- `POST` `/api/v1/billing/subscriptions/{organization_id}/downgrade`
- `POST` `/api/v1/billing/subscriptions/{organization_id}/suspend`
- `POST` `/api/v1/billing/subscriptions/{organization_id}/reactivate`
- `GET` `/api/v1/billing/usage/{organization_id}`
- `GET` `/api/v1/billing/usage/{organization_id}/records`
- `GET` `/api/v1/billing/limits/{organization_id}`
- `GET` `/api/v1/billing/estimate/{organization_id}`
- `POST` `/api/v1/billing/invoices/{organization_id}`
- `GET` `/api/v1/billing/invoices/{organization_id}`
- `GET` `/api/v1/billing/invoices/detail/{invoice_id}`
- `GET` `/api/v1/billing/credits/{organization_id}`
- `POST` `/api/v1/billing/credits/{organization_id}`

`branding_routes.py` — prefix `/branding`

- `GET` `/branding`

- `PUT` `/branding`
- `POST` `/branding/logo`
- `GET` `/branding/logo/current`

`dataset_routes.py` — prefix `/datasets`

- `GET` `/datasets`
- `GET` `/datasets/{dataset_id}`
- `GET` `/datasets/{dataset_id}/status`
- `GET` `/datasets/{dataset_id}/overview`
- `GET` `/datasets/{dataset_id}/preview`
- `POST` `/datasets/{dataset_id}/clean`
- `GET` `/datasets/{dataset_id}/clean/download`

`dax_routes.py` — prefix `/dax`

- `GET` `/dax/{dataset_id}/library`
- `POST` `/dax/{dataset_id}/generate`
- `POST` `/dax/explain`
- `POST` `/dax/optimize`
- `POST` `/dax/{dataset_id}/optimize`
- `POST` `/dax/detect-errors`

`evaluation_routes.py` — prefix `/api/v1`

- `POST` `/api/v1/evaluation/run`
- `GET` `/api/v1/evaluation/session/{session_id}`
- `GET` `/api/v1/evaluation/workflow/{workflow_id}`
- `GET` `/api/v1/evaluation/report/{evaluation_id}`
- `GET` `/api/v1/evaluation/export/{evaluation_id}`

`insight_routes.py` — prefix `/insights`

- `GET` `/insights/{dataset_id}`

- `GET` `/insights/{dataset_id}/decision-framework`
- `POST` `/insights/{dataset_id}/ask`

`intelligence_routes.py` — prefix `/intelligence`

- `GET` `/intelligence/{dataset_id}/data-insights`
- `GET` `/intelligence/{dataset_id}/ai-business-insights`
- `GET` `/intelligence/{dataset_id}/executive-storyboard`
- `GET` `/intelligence/{dataset_id}/domain`
- `GET` `/intelligence/{dataset_id}/regional`

`job_routes.py` — prefix `/api/v1/jobs`

- `POST` `/api/v1/jobs`
- `GET` `/api/v1/jobs/statistics`
- `GET` `/api/v1/jobs`
- `GET` `/api/v1/jobs/{job_id}`
- `DELETE` `/api/v1/jobs/{job_id}`
- `POST` `/api/v1/jobs/{job_id}/retry`

`knowledge_routes.py` — prefix `/api/v1`

- `POST` `/api/v1/knowledge/ingest`
- `POST` `/api/v1/knowledge/search`
- `GET` `/api/v1/knowledge/documents`
- `DELETE` `/api/v1/knowledge/{document_id}`

`monitoring_routes.py` — prefix `/api/v1`

- `GET` `/api/v1/monitoring/health`
- `GET` `/api/v1/ready`
- `GET` `/api/v1/live`
- `GET` `/api/v1/metrics`
- `GET` `/api/v1/system/status`

- `GET` `/api/v1/system/config`
- `GET` `/api/v1/system/dependencies`

`organization_routes.py` — prefix `/api/v1/organizations`

- `POST` `/api/v1/organizations`
- `GET` `/api/v1/organizations`
- `GET` `/api/v1/organizations/{organization_id}`
- `PUT` `/api/v1/organizations/{organization_id}`
- `DELETE` `/api/v1/organizations/{organization_id}`
- `POST` `/api/v1/organizations/{organization_id}/restore`
- `POST` `/api/v1/organizations/{organization_id}/invite`
- `POST` `/api/v1/organizations/invitations/accept`
- `POST` `/api/v1/organizations/invitations/decline`
- `GET` `/api/v1/organizations/{organization_id}/members`
- `DELETE` `/api/v1/organizations/{organization_id}/members/{member_user_id}`
- `POST` `/api/v1/organizations/{organization_id}/transfer-ownership`

`rag_routes.py` — prefix `/rag`

- `POST` `/rag/{dataset_id}/index`
- `GET` `/rag/{dataset_id}/status`
- `POST` `/rag/{dataset_id}/retrieve`
- `DELETE` `/rag/{dataset_id}/index`

`rbac_routes.py` — prefix `/api/v1`

- `GET` `/api/v1/roles`
- `GET` `/api/v1/permissions`
- `POST` `/api/v1/roles/assign`
- `POST` `/api/v1/roles/remove`
- `GET` `/api/v1/access/check`

`release_routes.py` — prefix `/api/v1/release`

- `GET` `/api/v1/release/benchmarks`
- `GET` `/api/v1/release/validation`
- `GET` `/api/v1/release/security/audit`
- `GET` `/api/v1/release/recovery`
- `GET` `/api/v1/release/performance`

`report_routes.py` — prefix `/report`

- `GET` `/report/{dataset_id}`
- `GET` `/report/{dataset_id}/export`

`sql_lab_routes.py` — prefix `/sql-lab`

- `GET` `/sql-lab/{dataset_id}/templates`
- `GET` `/sql-lab/{dataset_id}/history`
- `POST` `/sql-lab/{dataset_id}/query`
- `POST` `/sql-lab/{dataset_id}/generate`
- `POST` `/sql-lab/{dataset_id}/save`
- `POST` `/sql-lab/explain`
- `POST` `/sql-lab/optimize`
- `POST` `/sql-lab/detect-errors`

`storage_routes.py` — prefix `/api/v1/storage`

- `POST` `/api/v1/storage/upload`
- `GET` `/api/v1/storage/files`
- `GET` `/api/v1/storage/statistics`
- `GET` `/api/v1/storage/{object_id}`
- `GET` `/api/v1/storage/{object_id}/download`
- `DELETE` `/api/v1/storage/{object_id}`
- `POST` `/api/v1/storage/{object_id}/archive`
- `POST` `/api/v1/storage/{object_id}/restore`
- `POST` `/api/v1/storage/{object_id}/rollback`

- `POST` `/api/v1/storage/{object_id}/verify`

`system_routes.py` — prefix `/api/v1`

- `GET` `/api/v1/health`
- `GET` `/api/v1/version`
- `GET` `/api/v1/capabilities`

`theme_routes.py` — prefix `/themes`

- `GET` `/themes`
- `GET` `/themes/active`
- `POST` `/themes/active/{theme_name}`

`upload_routes.py` — prefix ``

- `POST` `/upload`

`visual_builder_routes.py` — prefix `/visual-builder`

- `GET` `/visual-builder/{dataset_id}/schema`
- `POST` `/visual-builder/{dataset_id}/render`
- `POST` `/visual-builder/{dataset_id}/register`

`workflow_routes.py` — prefix `/api/v1`

- `POST` `/api/v1/workflow/execute`
- `GET` `/api/v1/workflow/status/{execution_id}`
- `GET` `/api/v1/workflow/results/{execution_id}`
- `GET` `/api/v1/workflow/statistics`

`workspace_routes.py` — prefix `/api/v1/workspaces`

- `POST` `/api/v1/workspaces`
- `GET` `/api/v1/workspaces`
- `GET` `/api/v1/workspaces/{workspace_id}`
- `PUT` `/api/v1/workspaces/{workspace_id}`

- `DELETE` `/api/v1/workspaces/{workspace_id}`

- `POST` `/api/v1/workspaces/{workspace_id}/restore`

- `GET` `/api/v1/workspaces/{workspace_id}/members`

Best practices

- Prefer `/api/v1` gateway for new integrations

- Handle 401/403/429 (auth + rate limit + lockout)

- Paginate jobs/storage lists with `page` & `page_size`

- Use `/api/v1/release/validation` after deploy

Errors

Mapped via `backend/api/error_handlers.py` and service exceptions (`AuthError`, `JobError`, `StorageError`, etc.).

10_database / README.md

10 — Database Handbook

Persistence strategy

| Concern | Implementation | Notes |

| Engine/session | `backend/database/session.py` | Pool tuning; SQLite StaticPool for memory |

| Config | `backend/database/config.py` | DATABASE_URL |

| ORM models | `backend/database/models/` | auth, org, rbac, audit, knowledge, runtime |

| Repositories | memory + SQLAlchemy | `repositories/registry.py` switches |

| Migrations | Alembic (`alembic/`) | `e5e80b7071e5_initial_schema.py` present |

| Transactions | `database/transaction.py` | repository_context for multi-repo writes |

ER (logical)

erDiagram

USER ||--o{ SESSION : has

USER ||--o{ ORG_MEMBER : joins

ORGANIZATION ||--o{ WORKSPACE : contains

ORGANIZATION ||--o{ ORG_MEMBER : has

ROLE ||--o{ ROLE_ASSIGNMENT : granted

USER ||--o{ ROLE_ASSIGNMENT : receives

PERMISSION ||--o{ ROLE : includes

Exact columns: see SQLAlchemy models under `backend/database/models/`.

Object storage lifecycle (not SQL)

Upload → version → metadata → download/stream → archive/restore → rollback → retention/checksum verify.

Migration strategy

1. Backup data

2. `alembic upgrade head`

3. Validate `/api/v1/release/validation`

Commercial billing/API-key stores remain **in-memory** in v1.0 (KNOWN_ISSUES).

11_frontend / README.md

11 — Frontend Handbook

Entry & navigation

`frontend/streamlit_app.py` defines `_NAV_GROUPS`:

1. Home

2. AI Workspace (Analyst, Dataset Manager, Workflow, Knowledge, Evaluation, Sessions, Jobs, Storage…)

3. Data (Upload, Cleaning, Preview)

4. Analytics (Dashboard, Charts, Business Analysis, Pivot, Studio)

5. AI Legacy (Chat, Insights)

6. Reports (Reports, Storyboard, Location)

7. Advanced (SQL Lab, DAX, Settings)

8. Account (Login, Register, Profile, Password)

9. Administration (Orgs, Workspaces, Members, Invitations, Roles, Permissions)

10. Commercial (Billing, Usage, Subscriptions, API Keys, Admin…)

11. Operations (Health, Metrics, Status, Dependencies, Config)

Page modules (`frontend/app_pages/`)

Each `*_page.py` typically: check auth → call API client → render Streamlit widgets.

| Page module | Purpose |

| home_page | Landing / overview |

| auth_pages | Login/register/profile |

| ai_analyst_workspace_page | NL analysis |

| dataset_manager_page / dataset_page | Datasets |

| workflow_monitor_page | Workflows |

| knowledge_center_page | Knowledge |

| evaluation_dashboard_page | Evaluation |

| job_monitor_page | Jobs |

| storage_* / artifact_browser / dataset_versions | Storage lifecycle |

| dashboard_* / reports / storyboard / sql_dax / location | Analytics & reports |

| rbac_pages | Admin RBAC UI |

| billing_* / usage / subscription / apikey / admin / system_analytics | Commercial |

| system_health / metrics / application_status / dependency / configuration_viewer | Ops |

Session state

`frontend/utils/session_state.py`, `auth_state.py` — tokens, selected dataset, nav page.

Architecture

flowchart LR

Page --> Client[frontend/api clients]

Client --> FastAPI

Page --> Session[st.session_state]

Expected screenshots

See [12 Pictorial Evidence](../12_pictorial_evidence/README.md).

12_pictorial_evidence / README.md

12 — Pictorial Evidence

Automated UI screenshots were **not captured in this documentation generation run** (headless Streamlit screenshot automation: **Not verified** in repo tooling).

Manual screenshot checklist

For each item: capture PNG into `documentation/assets/screenshots/`.

| # | Title | Purpose | What must be visible | Proves |

| 1 | Login | Auth | Email/password form | Auth UI |

| 2 | Register | Auth | Registration form | Signup |

| 3 | Home / Dashboard | Analytics | KPI cards or overview | Core analytics |

| 4 | AI Analyst | AI | Query box + response | Analyst runtime |

| 5 | Workflow Monitor | Workflows | Execution list/detail | Workflow UI |

| 6 | Storage Manager | Storage | File list/upload | Storage |

| 7 | Knowledge Center | RAG/Knowledge | Ingest/search UI | Knowledge |

| 8 | Evaluation Dashboard | Quality | Scores | Evaluation |

| 9 | Job Monitor | Jobs | Job table/status | Async jobs |

| 10 | Organizations | Tenancy | Org list/create | Orgs |

| 11 | Roles/Permissions | RBAC | Role UI | RBAC |

| 12 | Settings | Branding/theme | Theme controls | Settings |

| 13 | System Health | Ops | Health status | Monitoring |

| 14 | Release Validation | Release | `/release/validation` via UI or API client | Release gates |

| 15 | API Docs | OpenAPI | FastAPI `/docs` | API evidence |

Caption template

> **Figure X — {Title}.** Captured from Data Bot AI v1.0.0 Streamlit UI. Demonstrates {Purpose}.

13_testing / README.md

13 — Testing Handbook

Statistics (verified at generation time)

| Metric | Value |

| `test_*.py` files under `tests/` | **125** |

| Full suite at v1.0.0 release gate | **634 passed** |

| Categories | unit, API, auth, orgs, rbac, jobs, storage, infra, billing, admin, apikeys, load, security, performance, reliability, release, database, repositories |

Testing pyramid (as practiced)

flowchart TB

L[Load / stress - tests/load]

S[Security / reliability / performance]

I[API + integration TestClient]

U[Unit service/model tests]

L --- S --- I --- U

What each area validates

| Folder | Validates |

| `tests/auth` | JWT, passwords, auth API |

| `tests/organizations`, `tests/rbac` | Tenancy + permissions |

| `tests/jobs` | Job lifecycle, queue, workers |

| `tests/storage` | Providers, versioning, API |

| `tests/infrastructure` | Config, logging, metrics, health |

| `tests/billing`, `apikeys`, `admin` | Commercial platform |

| `tests/load` | Concurrent users/jobs/API smoke |

| `tests/security` | Headers, sanitization, lockout |

| `tests/reliability` | Circuit, retry, timeout, shutdown |

| `tests/release` | Benchmarks, validation, E2E sample dataset |

| Root `tests/test_*.py` | AI, RAG, forecast, workflow, analytics contracts |

Why testing matters

Prevents regressions across a large service surface (~89 services) and supports the v1.0 production claim with measurable evidence.

14_sprints / README.md

14 — Sprint Timeline (6.9 → 8.7)

> Sprint numbering below follows product history referenced in code comments and release docs. Exact calendar dates: **Not verified**.

| Sprint | Goal | Architecture impact | Business value |

| 6.x–7.x | Analytics + AI foundations | Datasets, dashboards, insights, early AI | Core analyst product |

| 7.7–7.8 | Analyst runtime + `/api/v1` gateway | Workflows, evaluation, knowledge APIs | Production API surface |

| 8.0 | Authentication | JWT identity | Secure access |

| 8.1 | Orgs / workspaces / RBAC | Multi-tenant authorization | Team collaboration |

| 8.2 | SQLAlchemy persistence | Repositories + Alembic | Durable platform data |

| 8.3 | Jobs / queue / workers | Async execution | Long-running work |

| 8.4 | Storage lifecycle | Object storage abstraction | Dataset/artifact durability |

| 8.5 | Monitoring & config | Health, metrics, logging, docker | Operability |

| 8.6 | Commercial platform | Billing, usage, API keys, admin | Monetization controls |

| 8.7 | Production RC hardening | Performance, security, reliability, load tests, docs | Release readiness |

| **1.0.0** | Official release | Tag `v1.0.0`, maintenance branch | Production declaration |

Lessons learned

- Keep services behind interfaces (repos, storage, queue) for swap-outs

- In-memory MVP stores ship faster but must be documented as limitations

- Release gates (tests + E2E + checklists) beat feature count for production trust

15_learning / README.md

15 — Learning Handbook

Progression path using this repository as the lab.

Beginner

1. Python + pandas on `data/samples/sample_sales_data.csv`
2. Run FastAPI + hit `/health`
3. Run Streamlit and upload a dataset
4. Read `backend/main.py` router list

Intermediate

1. FastAPI routes → services pattern
2. JWT auth flow (`auth_service` + security)
3. SQLAlchemy session + Alembic
4. Repository pattern (`interfaces` vs memory vs SQLAlchemy)
5. Streamlit session state + API clients

Advanced

1. Workflow engine + jobs/queue/workers
2. Multi-agent / tool selection / memory / RAG services
3. Evaluation & validation pipelines
4. Middleware hardening (rate limit, CSRF, headers)
5. Observability (metrics/tracing/health)
6. Docker compose multi-process (API + worker + redis/postgres)

Topic index

Python, FastAPI, Streamlit, SQLAlchemy, Repository Pattern, DI-via-Depends, JWT, RBAC, Orgs, Jobs, Storage, Monitoring, Docker, Caching (`performance/cache`), Data Science, Statistics, Power BI analogues, ML/forecast modules, RAG, LLMs, Agents, Workflow engines — each maps to folders listed in Architecture + Folder docs.

16_interview / README.md

16 — Interview Preparation

Project discussion (STAR-ready)

Situation: Need a local-first AI analytics SaaS MVP.

Task: Deliver upload→insights→AI analyst→platform ops through v1.0.

Action: Layered FastAPI/Streamlit architecture with tests and hardening.

Result: Tagged `v1.0.0` with 634 passing tests and documented limitations.

Sample Q&A

System design

Q: How is multi-tenancy handled?

A: Organizations and workspaces with RBAC evaluation (`rbac_service.evaluate_access` / `has_permission`).

FastAPI

Q: Where is middleware configured?

A: `backend/main.py` — CORS, GZip, security headers, rate limit, CSRF, monitoring, auth context.

Security

Q: How are passwords stored?

A: PBKDF2-HMAC-SHA256 with salt (`password_service`), not plaintext.

AI

Q: How do you reduce hallucinations?

A: Retrieval (RAG/knowledge), tool-grounded workflows, and validation/evaluation services — not a hard guarantee.

Data

Q: Walk through upload.

A: `/upload` → upload/dataset services → metadata + processed artifacts → analytics endpoints.

Behavioral

Q: A limitation you shipped?

A: In-memory commercial stores and no payment gateway — documented in KNOWN_ISSUES, scheduled on ROADMAP.

More prompts: Python GIL vs workers, idempotent jobs, circuit breakers, pagination, CORS credentials vs `*`, Alembic vs create_all.

17_portfolio / README.md

17 — Portfolio Package

One-liner

Data Bot AI v1.0 — local-first AI analytics SaaS: datasets → dashboards → AI analyst workflows → multi-tenant platform ops.

Resume bullets

- Built FastAPI + Streamlit analytics platform with 89 service modules and `/api/v1` gateway

- Implemented JWT auth, org/workspace RBAC, async jobs, object storage, and monitoring

- Delivered AI analyst runtime with planning, tools, memory, RAG, and evaluation services

- Hardened production RC (caching, rate limits, CSRF, circuit breakers) and released `v1.0.0` with 634 automated tests

LinkedIn project blurb

Data Bot AI is an end-to-end analytics and AI assistant platform. It combines classical BI (profiling, KPIs, DAX/SQL labs, exports) with an AI analyst runtime and production platform features (auth, RBAC, jobs, storage, billing controls).

Case study structure

1. Problem → fragmented analytics + AI experimentation

2. Solution → layered MVP architecture

3. Challenges → state management, AI reliability, multi-tenant security

4. Solutions → services + validation + middleware + tests

5. Impact → shippable v1.0 with evidence

6. Next → ROADMAP.md

GitHub overview

Point reviewers to: `README` (root), `documentation/`, `release/v1.0/`, tag `v1.0.0`, OpenAPI `/docs`.

18_release / README.md

18 — Release Documentation Index

Canonical release artifacts also live at repo root / `docs/release/` / `release/v1.0/`.

| Doc | Location |

| CHANGELOG | `/CHANGELOG.md` |

| ROADMAP | `/ROADMAP.md` |

| RELEASE NOTES | `/RELEASE_NOTES.md` + `docs/release/RELEASE_NOTES.md` |

| KNOWN ISSUES | `/KNOWN_ISSUES.md` |

| Architecture Guide | `docs/release/ARCHITECTURE_GUIDE.md` |

| Deployment Guide | `docs/release/DEPLOYMENT_GUIDE.md` |

| API Guide | `docs/release/API_GUIDE.md` |

| Security Guide | `docs/release/SECURITY_GUIDE.md` |

| DR Guide | `docs/release/DISASTER_RECOVERY_GUIDE.md` |

| Checklists | `release/v1.0/*` |

Copies for handbook convenience:

18_release / CONTRIBUTING.md

CONTRIBUTING

Branches

- `main` — new features

- `release/v1.0` — bug fixes, security patches, docs only

Rules for release/v1.0

No architectural changes, no breaking API changes, no new product features.

Dev loop

pip install -r requirements-dev.txt

pytest -q

python -m compileall backend frontend

Docs

Update `documentation/` when behavior changes. Do not invent features.

19_formats / README.md

19 — Output Formats

| Format | How |

| Markdown | This `/documentation` tree |

| Mermaid | Embedded in markdown (GitHub/compatible renderers) |

| PDF / DOCX / PPTX | `python scripts/export_handbook.py` |

PlantUML: optional; Mermaid is the primary diagram DSL used here.

20_standards / README.md

20 — Documentation Standards

1. Professional tone; short paragraphs; tables for inventories

2. Diagrams for flows (Mermaid)

3. Examples tied to real paths (`backend/...`)

4. Never invent functionality

5. Label gaps as **Not verified**

6. Prefer evidence: tests, routes, services, release tags

7. Keep handbook in sync with `KNOWN_ISSUES.md` and `ROADMAP.md`